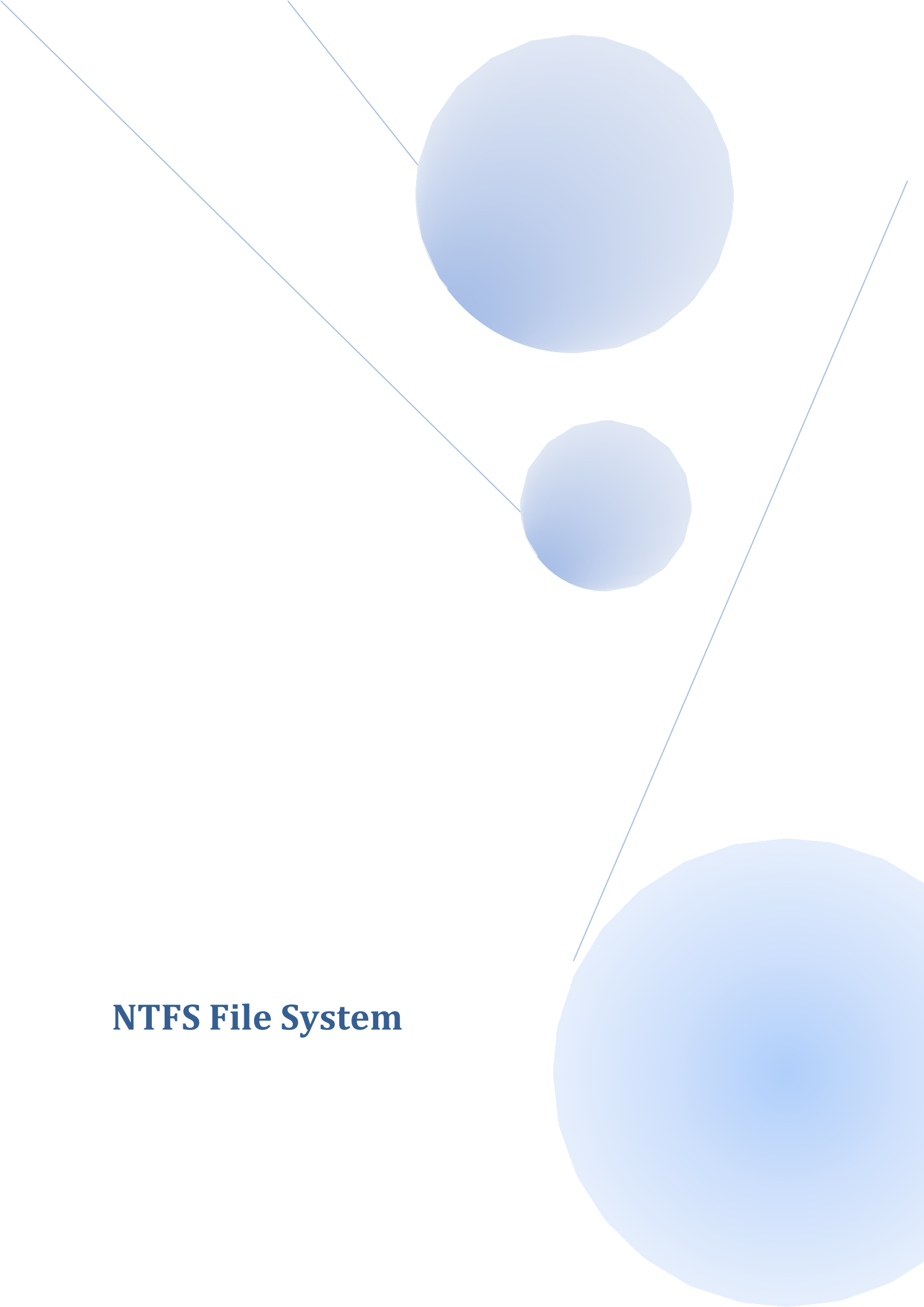


The background features three blue circles of varying sizes and three thin blue lines. One line connects the top-left corner to the top circle. Another line connects the top-right corner to the top circle. A third line connects the bottom-right corner to the bottom circle. The text 'NTFS File System' is positioned in the lower-left area.

NTFS File System

1. Introduction to NTFS
2. NTFS Concepts
3. Master File Table
4. Attributes
5. Metadata Files
6. Representation of Files
7. Representation of Directories
8. Recovery of deleted files and folders
9. Advanced NTFS Features

Why NTFS?

Ordinary users were comfortable with FAT family of file systems, but...

FAT Cannot

- Support high storage demand
- Provide security to users
- Ensure reliability

During the development of Windows NT operating system, Microsoft decided to make it a very reliable high performing operating system by adding features like security and recoverability. But FAT was too simple. FAT cannot support large sized files. There was no provision for incorporating security information into its simple data structures. So Microsoft designed a new file system from scratch. This file system is named New Technology File System.

NTFS Design Objectives

- Reliability
- Security
- Efficient Data Storage

New Technology File System (NTFS), introduced by Microsoft with the Windows NT operating system to handle data storage and management in a high end networked corporate environment. It is designed to quickly perform standard file operations such as read, write, and search. Main objectives for developing this file system includes reliability, indexing and security.

Reliability: The file system records the transactions and if there is any failure, system can roll back to a previous consistent state.

Security and Access control: File system can control who all users have access to specific files/directories. Security feature is provided by the operating system and NTFS supports it with its own metadata files (\$Secure file) and data structures.

Indexing: Files and folders are kept in a sorted manner within a directory. This enables quick file searching and listing.

Hard Links: Same file can have different names in different directories. The directory entries may point to the same MFT entry.

Efficient Data Storage: Contents of small files are stored within NTFS main data structure, Master File Table and so no disk access is required to read small files once it gets the MFT record representing the file.

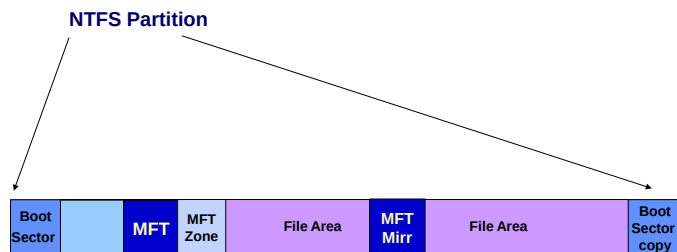
- The main database is Master File Table(MFT)
 - Everything is treated as a file
 - Every file is treated as a set of attributes.
 - Every file has a record/entry in MFT

NTFS volume consists of files and unallocated free space. Each cluster within a volume can be identified by **64 bit** logical cluster no. Cluster size may vary depending on the drive size. The data structure used by NTFS is the Master File Table (MFT). MFT contains an entry/record for each file and folder present in the partition. Everything in the volume is treated as a file and everything related to a file is treated as an attribute. The filename, file contents are all treated as attributes.

An NTFS Partition contains

- Boot Sector
- Master File Table(MFT)
- MFT Zone
- Metadata files
- Copy of MFT (MFT Mirror)
- Boot Sector Copy

A volume formatted using NTFS contains boot sector, reserved sectors, MFT, a copy of MFT, a copy of boot sector and user file area. Cluster is the basic allocation unit. Each cluster consists of a set of sectors. The first cluster contains the Boot sector. Then a few clusters are reserved by the file system. MFT is stored after the reserved sectors. A copy of the first 4 records of MFT (MFT Mirror) is stored at the exact middle of the volume. This is known as MFTMirr. The last sector stores a copy of the boot sector.



Layout NTFS Partition

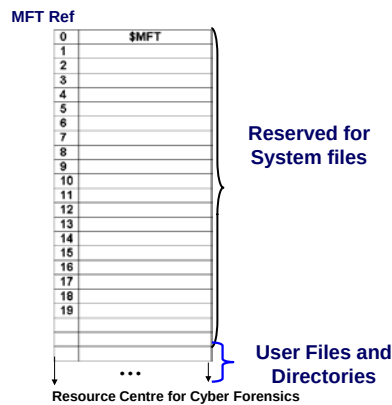
NTFS Boot Sector

- Bytes per sector.
- Sectors per Cluster
- Total sectors.
- Logical cluster number for the MFT
- Logical cluster number copy of the MFT
- MFT Record Size
- Index Block Size

The boot sector contains the location of MFT and MFTMirr, cluster size, MFT Record Size and Index block size (Index blocks are allocated for storing directory entries).

- Main data structure of NTFS
- Equal sized records
- Every file/directory has an entry
- Each record contains a set of attributes
- Each record is uniquely identified by MFT Reference Number

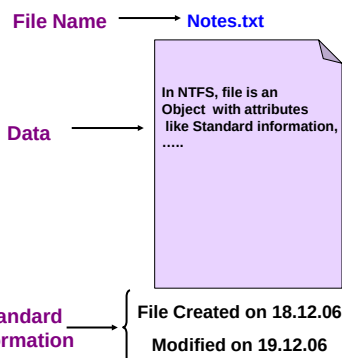
Master file table is the main data structure used by NTFS. This table stores all information about all the files and folders in the volume. MFT is treated as a file containing a number of records/entries. All the records are of same size. The size of an MFT record is specified in boot sector. MFT record size may vary from 1KB to 4 KB, usually 1024 bytes. Each record is numbered starting from 0. This number is known as MFT Reference number.



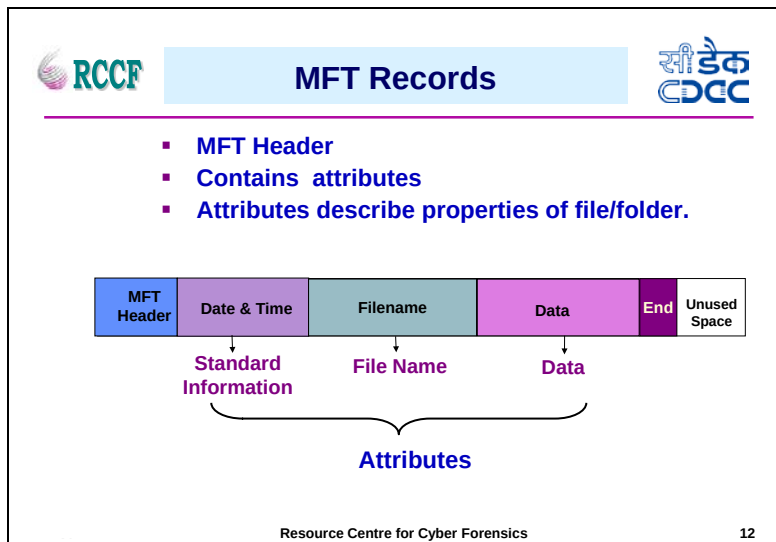
Each MFT record or entry contains a set of attributes associated with a file or folder. A record is created within the MFT when any file or directory is created. In MFT each record is numbered, but this number is not stored as such in the table. The first record has the number zero, next record is one and so on. In the diagram, MFT Ref column is shown to make it easy to understand. Remember that in a real MFT there is no column to represent the MFT Reference number.

First 16 records represent files containing information about the file system. These files are known as metadata files.

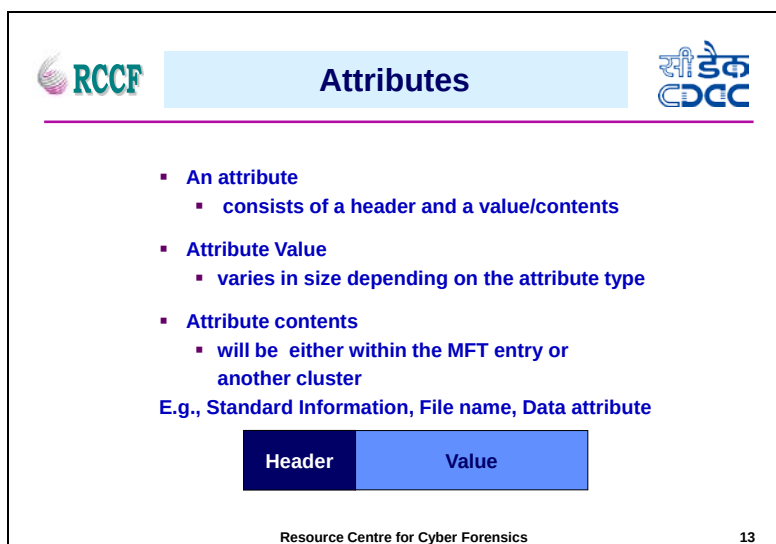
- Everything in a file is an attribute



An NTFS file is represented as a set of attributes. Common attributes are filename, creation/modification and last access time etc. An MFT Record is allocated whenever a file/directory is created.



The NT file system views each file as a set of attributes. Elements like filename, security information, data are attributes. For every file within the volume a record in MFT is allocated. The first 42 bytes of this MFT record are used to store MFT Header. The remaining bytes are used to save attributes associated with files. MFT Header contains flag fields, offset to the start of next attribute etc. The end of a Master File Table record is identified by an end marker byte sequence (ff ff ff ff). Each attribute in NTFS has a type and a name.



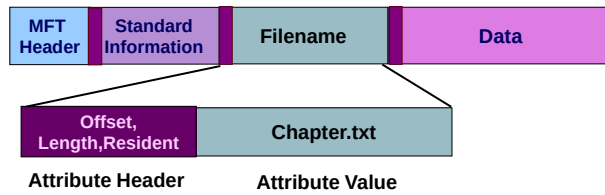
Each attribute present in the MFT record has a header and a value. The header indicates the type of the attribute, length of attribute etc. NTFS defines fourteen system defined attributes including standard information, filename, data, security descriptor, index root, index allocation. The name of an attribute starts with the symbol '\$'.

- Identifies type, name and size
- Indicates whether resident or non resident
- Identifies whether named or no named

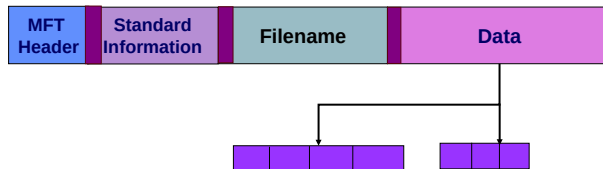
An attribute has a header and a value. The header contains information like attribute type, attribute id, attribute length, name length, offset to attribute name etc. The value represents the particular value of the attribute. For example, the value of DATA attribute for a file contains the data/contents of the file. Also, an attribute can have a name. If it has a name, it is 'named' attribute, otherwise 'no named' attribute. DATA can be named or no named. Since the MFT record is of fixed size, sometimes the attribute value may not fit into the MFT record. In such cases, the information for locating the attribute value will be stored after the header. Thus based on the location of attribute value, attributes are classified as resident attributes and non-resident attributes.

- Stores its contents within the MFT Record.
 - Can be converted to non resident
 - An attribute will be made resident if can fit within the MFT record.
- E.g. : Standard information, index root

If possible, the attribute's data will be stored in the MFT Record itself. If the value of an attribute is stored within the MFT record, it is known as resident attribute. For example, filename, index root attribute (used by directory entry in MFT) are resident.



- Non resident attributes are stored outside MFT.
- An attribute is made non-resident if its value exceeds the available space in MFT
- The cluster list/run where the attribute is stored, will be written after the attribute header.



If the value of an attribute is stored in clusters outside the MFT record, it is known as non-resident attribute. The value portion of a non-resident attribute contains the cluster run (data run) information. An attribute is made known resident if its value cannot be stored in the MFT Record. For example, the DATA attribute is usually made non-resident if the size exceeds around 700 bytes. Thus files of size less than 700 bytes (approx.) can be located within the MFT. Large files will be stored as non-resident attribute. *So in NTFS, small files can be accessed quickly.*

- File Creation Time
- Modified Time
- MFT Modified Time
- Accessed Time
- Flag fields

This attribute exists for all files and directories. This attribute has a type identifier 16. It gives the following timestamps, flags, security and quota information.

- File Creation Time: The time when the file was created.
- Modified Time: The time when the \$DATA and the \$INDEX attributes are modified.
- MFT Modified Time: The time when the metadata of the file was modified.
- Accessed Time: The time when the contents of the file were last accessed.

The flag fields indicate whether the file is encrypted, compressed or sparse or read only etc.

- Contains file name
- Length of file name
- File Reference to the Parent Directory
- At least one instance in every MFT record
- Flag Fields
- Contains timestamp information

Every file or directory in an NTFS volume will have at least one \$FILE_NAME attribute in its MFT entry. This attribute contains the name of the file in Unicode format. This attribute has a type identifier of 48 and the length varies depending on the length of filename. A file can have more than one instance of this attribute for storing short filename and long file name etc. Flag fields in this attribute also tell whether the file is read-only, sparse, compressed or encrypted as the flags in \$STANDARD_INFORMATION.

- Represents data/contents of a file
- Contains size of the attribute
- Resident if data size fits within MFT otherwise non-resident
- Usually No named attribute

This attribute is used to store the data. Its type identifier is 128 and its length can vary from 0 to any size. Every file will have one unnamed \$DATA attribute.

- To store the allocation status of entries in MFT
- Each bit corresponds to one MFT entry
 - Bit = 1 then MFT entry is allocated
 - Bit = 0 then MFT entry is free

\$BITMAP

This attribute is used to store the allocation status of entries in \$MFT. It has a type identifier 176. Each bit in the value field of this attribute corresponds to one entry. If it is set to one it means that the corresponding entry is allocated otherwise the entry is free. In \$MFT (entry 0 of MFT), this attribute is used to indicate the allocation status of MFT entries. If a bit is zero, it means the corresponding MFT record is free and may contain information of a deleted file.

- If too many attributes, an Attribute List Attribute is created
- The attributes within the attribute list will be placed at other MFT records
- The MFT record where attribute list is created is known as Base MFT Entry
- The record where the attributes mentioned in the attribute list are stored, is known as Extent MFT

When a file or directory contains too many attributes to fit into an MFT record, and attribute list attribute is created. This attribute is used to store a list of entries and has a type identifier 32. Each entry will represent an attribute. These attributes represented by the attribute list attribute will be stored in additional MFT records. The MFT record where attribute list attribute is created is known as **Base MFT** and the additional MFT record used to store the attributes are known as **Extent MFT**.

- Contain file system information
- Some of the Meta data files are
 - Master File Table (\$MFT)
 - Master File Table Mirror (\$MFTMirr)
 - Bitmap File (\$Bitmap)
 - \$.
 - \$Boot File

Metadata files store data about the file system. These files are hidden from view on the NTFS volume. Meta data files are placed on the volume by the Format utility.

- **MFT**
 - MFT entry for MFT itself
 - Data attribute specifies the layout of MFT
 - Bitmap attribute indicates the allocation status of MFT records
- **MFTMirr**
 - A copy of MFT, so MFT Mirror
 - Is located at the center of the volume
 - Contains first four records of the MFT representing MFT, MFTMirr, Log File and Volume

The Master File Table itself is treated as a file and the first entry, Entry 0 is used to store its attributes. The Data attribute describes the layout of Master File Table. The bitmap attribute value indicates the allocation status of MFT entries.

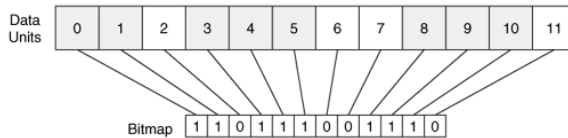
A copy of first four records of MFT is kept at the middle cluster of the volume. This avoids any MFT read error due to single sector failure.

- **Boot**
 - Contains boot sector
 - Details at MFT entry 7
 - The Boot sector gives
 - the size of each cluster
 - the number of sectors in the file system
 - the starting cluster address of the MFT,
 - and the size of each MFT entry

- **BadClus File**
 - This file points to the bad clusters in the volume
 - Bad clusters will be a part of this metadata file
- **. File**
 - This file points to the root directory
 - Searched first during file/folder lookup
 - To retrieve the full path name of a file or directory

■ Bitmap File

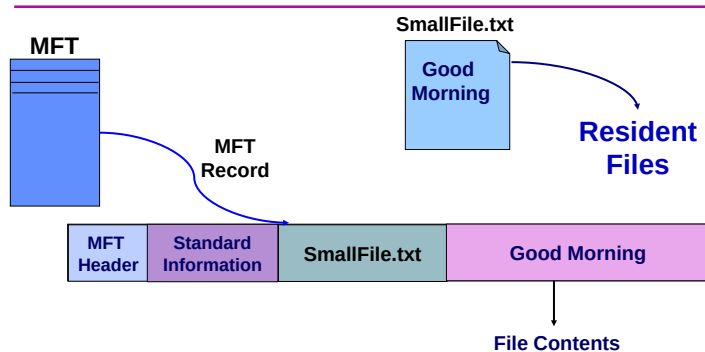
- Indicates the allocation status of all clusters within the volume
- Within the file, a bit is used to represent a cluster
- If the bit is set to one, that cluster is allocated, otherwise free



Resource Centre for Cyber Forensics

27

The \$Bitmap file indicates the allocation status of all clusters within the partition. One bit represents one cluster, first bit represents the first cluster, second bit represents the second cluster and so on. If the bit is 1 it means that the corresponding cluster is allocated. If the bit is zero it means the corresponding cluster is not allocated, that is it is free. In the figure given as example, the third, seventh, eighth and 12th bits are zero and so 3rd, 7th, 8th and 12th clusters are free and the remaining clusters are allocated. While creating a file, this bitmap file is used to identify available clusters. When the user deletes a file, the clusters used by the file are made available or freed by setting the corresponding bits to zero in this bitmap file.



Representation of Small Files

Resource Centre for Cyber Forensics

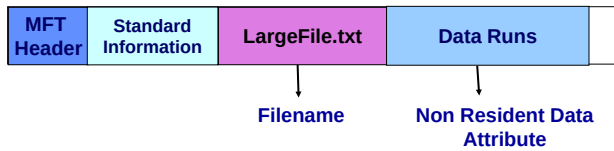
28

Representation of Small Files

The time stamp information about of the file will be initialized and saved as standard attribute and the filename will be saved in Filename attribute. The Standard Information and Filename attributes are always resident. The data attribute represents the contents of the file. In the case of a small file with data size that can fit within the MFT record's unused area, the data attribute is made resident. So access time is low for small files. Since MFT Reserved space is used instead of using a free cluster, the available free disk space will be same as before the file creation.

Large Files

- If contents do not fit within the MFT Record then contents are stored as non resident attributes



Representation of Large Files

Resource Centre for Cyber Forensics

29

Representation of large files

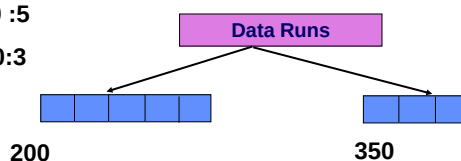
Assignment of time stamp information and filename are same as in small files. But the data attribute is made non-resident as the contents cannot fit within the MFT record. Contiguous free clusters are allocated to store the data and the cluster numbers are saved as data runs after the header of data attribute.

- Data stored outside MFT
- Free contiguous clusters are chosen
- Data Runs are cluster lists where data is stored
- Extra MFT records may be needed

Data Runs

200 :5

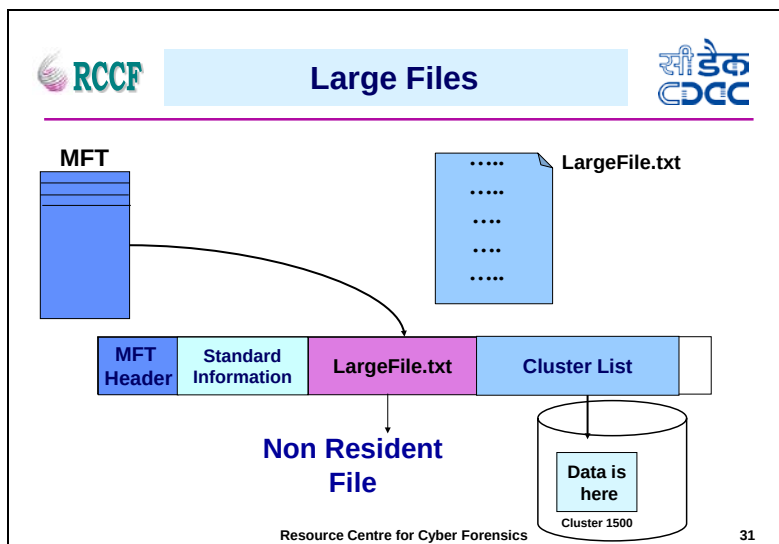
350:3



Resource Centre for Cyber Forensics

30

Data runs gives the list of clusters where the value of an attribute is stored. For a fragmented file, there may be more than one data run for the \$DATA attribute. Each data run contains the start cluster number and the number of clusters beginning from that cluster. If there are too many cluster lists to be saved then additional MFT records will be allocated.



Representation of large files

Assignment of time stamp information and filename are same as in small files. But the data attribute is made non-resident as the contents cannot fit within the MFT record. Free clusters are allocated to store the data and the cluster numbers are saved as cluster list in the header of data attribute.

The diagram illustrates the structure of a directory in NTFS. It shows an MFT entry for a directory. The MFT entry is divided into four parts: 'MFT Header', 'Standard Information', 'INDEX_ROOT', and 'INDEX_ALLOCATION'. The 'INDEX_ROOT' attribute is described as 'Resident' and 'Contains index entries'. The 'INDEX_ALLOCATION' attribute is described as 'Non Resident', 'Stores index entries in index blocks', and 'Records cluster locations of index blocks'. The diagram is titled 'Directory' and includes logos for RCCF and सी डेक CDC.

Representation of directory

MFT record header contains a flag (means a bit- a binary value), to indicate whether an MFT record represents a directory or a file. If the flag bit is set to one it means that the MFT record represents a directory. On the other hand, if the bit is set to zero, it represents a file.

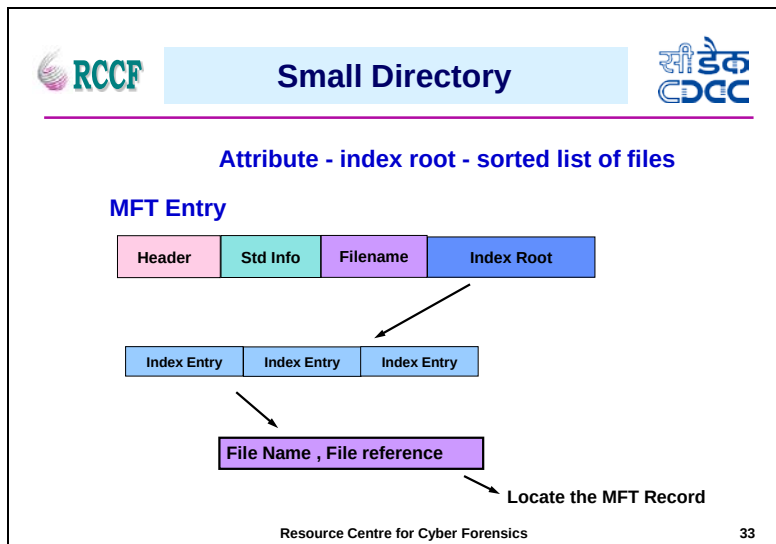
An MFT record representing directory will always contain a standard information attribute, a file name attribute (to store directory name) and an index root attribute. If the directory contains too many entries it will have an index allocation attribute also. Usually there will not be any data attribute associated to a directory. But if there is any alternate data stream then directory will have a named data attribute, the name of the attribute is the name of the alternate data stream.

\$INDEX_ROOT

Every directory has an \$INDEX_ROOT attribute that contains information about the files and subdirectories that are located in it. It is always resident and has a type identifier of 144 and name '\$I30'.

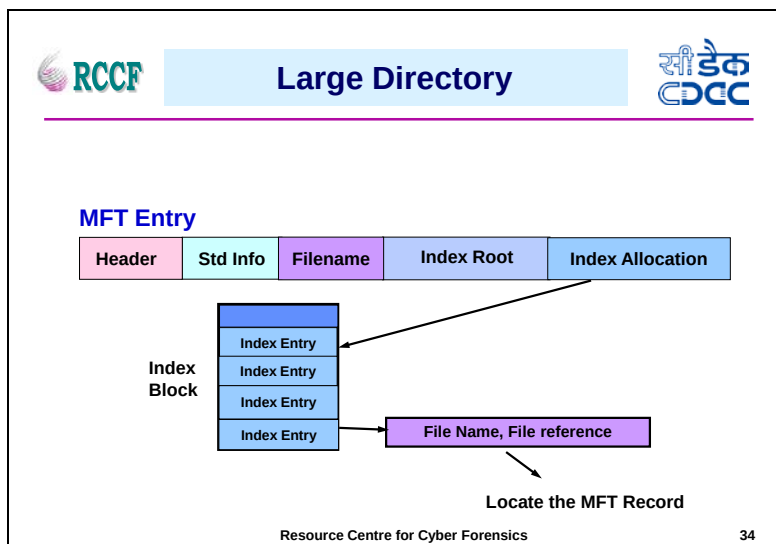
\$INDEX_ALLOCATION

If the directory is too large then it will have an \$INDEX_ALLOCATION attribute to store file entries. This attribute has a type identifier 160 and name '\$I30'.



Representation of Small Directory

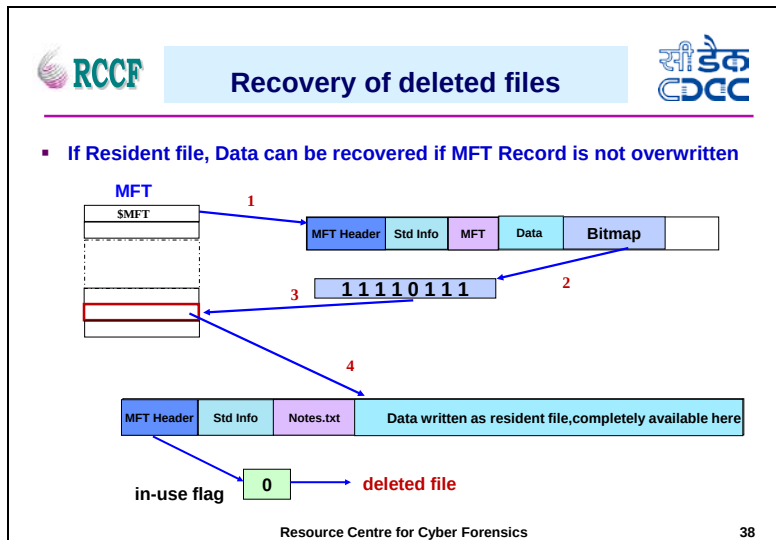
Index root attribute will contain index entries. Each index entry contains a File name and it's MFT Reference Number.



- Find out a free MFT entry for the new file using the BITMAP attribute value of MFT.
- When a free entry is found, set the corresponding bit in the BITMAP attribute value to indicate that it is allocated
- Set the in-use flag bit in the MFT entry to 1
- Write the STANDARD_INFORMATION and FILENAME attributes

- Check whether data attribute can be made resident or non resident
- If non resident then allocate the required number of clusters using Bitmap File, update the Bitmap
- From the root directory, locate the parent directory and add a new index entry within it

- Locate the index entry of the file by searching through the root directory.
- Delete index entry from the parent folder
- Update the BITMAP attribute of MFT to indicate that this entry is now free
- Reset the in-use flag bit in the MFT Header
- Free the clusters by resetting the corresponding bits in the Bitmap file.

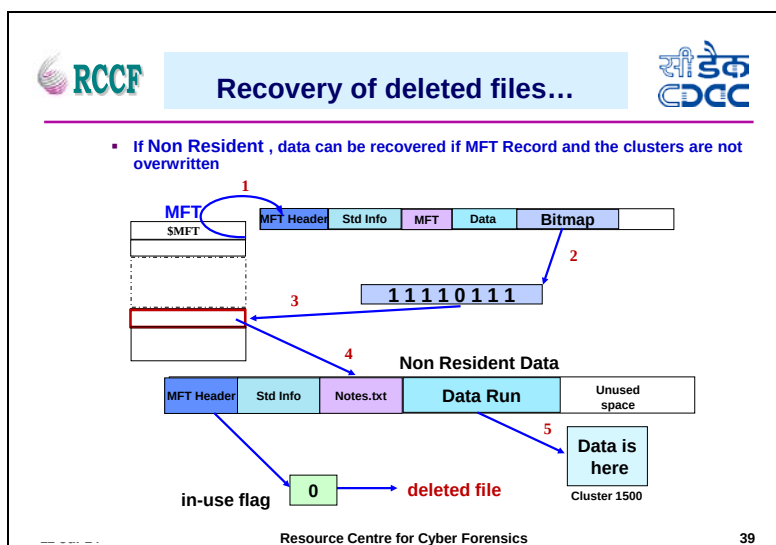


Recovery of Deleted Files

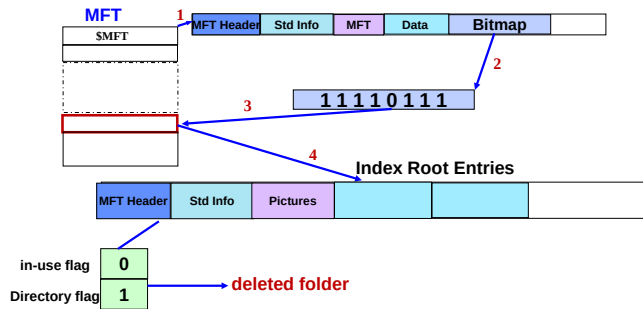
To recover all the deleted files and folders within the partition we check the allocation status of the bitmap attribute. A 0 bit in bitmap indicates that corresponding entry is free. The MFT Header's in-use flag indicates that the record is not in-use. We can read the attributes from such records to recover details about deleted files/folders.

Steps

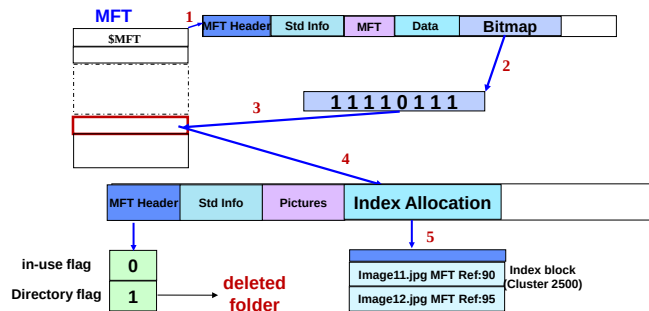
- Read the contents of MFT file's bitmap attribute
- Check for bits set to 0.
- If we get a 0 bit it means corresponding MFT entry is not yet allocated and may contain details of a deleted file.
- Go to the Unallocated entries and check whether it represents a file/ folder by checking the directory flag bit
- If it represents a file then read the Standard Information, filename and data attributes.
- If the data attribute is resident then data can be recovered completely
- If the data attribute is nonresident then the clusters represented by the data run may contain deleted file's content if those clusters have not yet been allocated to another file.



- If directory entries were within the MFT (Index Root) then directory contents can be recovered if MFT is not overwritten



- If directory entries were within index blocks then recovery is possible only if the blocks are not overwritten



Recovery of deleted folders

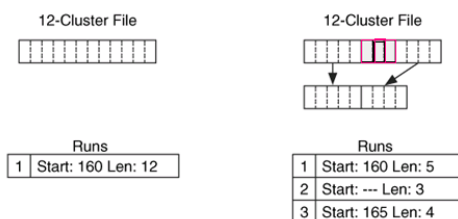
An MFT entry representing a deleted folder can be identified by directory flag set with 1 and an in-use flag with value 0. If the directory's entries are stored within an Index root attribute all the entries can be recovered. But when Index allocation attribute is used we will get the index block entries only if those clusters are not overwritten.

- To save storage space, data attribute can be made sparse, if it has clusters of zeros
- Clusters containing zeros are written as a special data run
- Cluster run will be specified by size without a starting cluster

Sparse Data

Sometimes, the data attribute may contain clusters of zeros. Instead of storing such attributes completely, only relevant data (non zero) can be stored on the disk. This type of data is known as **a sparse data** and such files are known as sparse files.

NTFS handles read operations on sparse files by returning allocated data and sparse data. While reading NTFS fills the memory buffer with zeros on encountering sparse clusters. If you copy or move a sparse file to a FAT or a non-Windows 2000 NTFS volume, the file is built to its originally specified size. If the required space is not available, the operation does not complete.

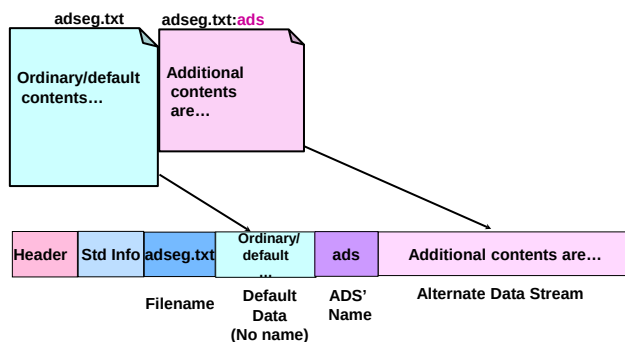


```

F:\>echo Default contents are unnamed data attribute>adseg.txt
F:\>echo Additional contents are named data attribute>adseg.txt:ads
F:\>more <adseg.txt:ads
Additional contents are named data attribute
F:\>more <adseg.txt
Default contents are unnamed data attribute

```

In NTFS file system, we can have streams other than usual data stream. These data streams are known as Alternate Data Streams (ADS). A file can have multiple ADSs. All alternate data streams are named and default data stream is unnamed. Maximum stream length is 2^{64} bytes. These ADSs are used to store information about the file like keywords, temporary data, thumbnail images etc.



Here, ResADS.txt is a file with a few characters. Since the contents can fit within the MFT record, it is made resident data. When an alternate data stream is created with the name 'Stream1', it is stored as resident data attribute with the name 'Stream1'.

- Data attribute can be compressed using an algorithm(modified LZ77)
- Only non resident attributes can be compressed
- The presence of compressed attribute is indicated by the flag in the non resident header
- Data are broken up into smaller units called compression units.Compression unit size is specified in the attribute header
- Compression is performed only if it could save at least one cluster

- Data attributes can be encrypted
- A File Encryption Key(FEK) is generated
- FEK is used to encrypt the data using DESX algorithm
- An attribute, \$Logged_Utility_Stream is added
 - Contains FEK in an encrypted state
 - Data Decryption fields and Data Recovery fields
- A temporary file EFS0.tmp is created
 - Contains the plaintext version of the file being encrypted
 - After encryption file is deleted but contents are not wiped

When an NTFS \$DATA attribute is encrypted, its contents are encrypted with a symmetric algorithm called DESX. One random key is generated for each MFT entry with encrypted data, and it is called the file encryption key (FEK). If there are multiple \$DATA attributes in the MFT entry, they are all encrypted with the same FEK.

The FEK is stored in an encrypted state in the \$LOGGED_UTILITY_STREAM attribute. The attribute contains a list of data decryption fields (DDF) and data recovery fields (DRF). A DDF is created for every user who has access to the file, and it contains the user's Security ID (SID), encryption information, and the FEK encrypted with the user's public key. A data recovery field is created for each method of data recovery, and it contains the FEK encrypted with a data recovery public key that is used when an administrator, or other authorized user, needs access to the data.

In fact, there is a small flaw in the NTFS design because it creates a temporary file named EFS0.TMP and it contains the plaintext version of the file being encrypted. After the OS finishes encrypting the original file, it deletes the temporary file, but the contents are not wiped. Therefore, a plaintext version of the file exists, and recovery tools might be able to recover the file if its MFT entry has not been reallocated. The swap space or page file might also provide copies of unencrypted data. It has been reported that if the administrator, domain controller, or other account that is configured as the recovery agent is compromised, any file can be decrypted because that account has access to all files .[Reference: 'File System Forensic Analysis' by Brian Carrier].

- www.ntfs.com
- www.linux-ntfs.org
- File System Forensic Analysis By Brian Carrier
- www.pcguides.com